

A PROJECT REPORT ON

Medical Image Analysis Using Transfer Learning

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Rucha Sahare
Sakshi Satpute
Chaitralee Sane

Exam No: B400050573
Exam No: B400050582
Exam No: B400050578



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

Medical Image Analysis Using Transfer Learning

Submitted by

Rucha Sahare
Sakshi Satpute
Chaitralee Sane

Exam No: B400050573
Exam No: B400050582
Exam No: B400050578

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. R. S. Paswan** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. R.S.Paswan
Internal Guide
Dept. of Computer Engg.

Dr. G.V.Kale
Head
Dept. of Computer Engg.

Dr. S.T.Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date:25/04/25

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Medical Image Analysis using Transfer Learning**’.*

*We would like to take this opportunity to thank **Prof. R. S. Paswan** for giving us all the help and guidance we needed. We are really grateful to her for her kind support. Her valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of the Computer Engineering Department, PICT, for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.*

Rucha Sahare
Sakshi Satpute
Chaitralee Sane
(B.E. Computer Engg.)

ABSTRACT

This study compares and implements three deep learning-based object detection models, YOLOv5, YOLOv8, and YOLOv11, for the purpose of detecting leukemia cells in blood smear samples using image analysis via light microscopy. The primary aim of the study was to investigate the performance of the three models to assess their worth in targeting abnormal leukemia cells in peripheral blood, with the metrics of precision, recall, mean Average Precision (mAP) and inference time considered using the ALL-IDB1 dataset. YOLOv5 was selected to deploy the model for real-time use, primarily because it is lightweight, and has a high recall—theoretically at 1.0 recall, which is ideal in order to limit false negatives in medical diagnosis; despite YOLOv11 yielded higher accuracy and recall. A Flask-based web application was developed for ease of use that utilized supported standard upload of images, displayed the prediction on the input image, and monitored real-time detection while integrating YOLOv5 for the detection of leukemia cells in microscopic blood smear images. Through careful testing and system integration, a conclusion was outlined that considered the potential of YOLObased models in improving early and accurate diagnosis of leukemia, while providing a novel and exciting platform for diagnosis/image analysis via real time methods.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	3
1.4	Methodologies of Problem solving	3
2	Literature Survey	6
3	Software Requirements Specification	9
3.1	Assumptions and Dependencies	10
3.2	Functional Requirements	10
3.2.1	Uploading and Image Detection System Capability 1	10
3.2.2	System Aspect 2: Results Visualization	11
3.2.3	System Features: Accessibility and User Involvement	11
3.3	External Interface Requirements	12
3.3.1	User Interfaces	12
3.3.2	Hardware Interfaces	12
3.3.3	Software Interfaces	12
3.3.4	Communication Interfaces	13
3.4	System Requirements	13
3.4.1	Database Requirements	13
3.4.2	Software Requirements (Platform Choice)	13
3.4.3	Hardware Requirements	13
3.5	Project Development Process	14
4	System Design	15
4.1	System Architecture	16
4.2	Data Flow Diagrams	17
4.3	UML Diagrams	18
4.4	Sequence Diagram	19

5	Project Plan	20
5.1	Project Estimate	21
5.1.1	Reconciled Estimates	21
5.1.2	Project Resources	21
5.2	Risk Management	22
5.2.1	Risk Identification	22
5.2.2	Risk Analysis	22
5.2.3	Overview of Risk Mitigation, Monitoring, and Man- agement	23
5.3	Project Schedule	24
5.3.1	Project Task Set	24
5.3.2	Task Network	25
5.4	Team Organization	25
5.4.1	Team Structure	25
5.4.2	Management Reporting and Communication	25
6	Project Implementation	26
6.1	Overview of Project Modules	27
6.2	Tools and Technologies Used	27
6.3	Algorithm Details	28
7	Software Testing	29
7.1	Type of Testing	30
7.2	Test Cases & Test Results	30
8	Results	31
8.1	Outcomes	32
8.2	Screen Shots	32
9	Conclusions	34
9.1	Conclusion	35
9.2	Future Work	35
9.3	Applications	36
10	Appendix	37
	Appendix A	38
10.1	Appendix C	41
11	References	42

List of Figures

4.1	Model Architecture	16
4.2	Data Flow Diagram	17
4.3	UML Diagram	18
4.4	Sequence Diagram	19
8.1	Screenshot 1: Image Upload Interface	32
8.2	Screenshot 2: Detection Results with Bounding Boxes	33
8.3	Screenshot 3: Detection Statistics and Insights	33

CHAPTER 1

INTRODUCTION

1.1 Overview

This project offers a comprehensive study of the use of the YOLOv5, YOLOv8, and YOLOv11 deep learning-based object detection models for the real-time identification of leukemia cells in microscopic blood smear images. Using the ALL-IDB1 dataset, the main objective was to assess and contrast how well these models performed on important measures including accuracy, recall, mean Average accuracy (mAP), and inference time.

Each model was trained and evaluated under consistent conditions following the dataset's preprocessing and augmentation. YOLOv5 was chosen for deployment because of its high recall, lightweight design, and simplicity of integration, even though YOLOv11 demonstrated the best overall accuracy and recall. To illustrate real-time leukemia detection, a Flask-based web application was created that allows users to upload photos, view findings, and instantaneously obtain detection data.

This study demonstrates how YOLO-based structures can help with automated and early medical diagnosis, especially in the field of hematology. Additionally, it demonstrates the actual viability of AI in clinical operations by establishing a complete pipeline from model training to real-time deployment.

1.2 Motivation

Leukemia is a serious illness that requires prompt and precise diagnosis for successful treatment. Conventional diagnostic techniques that rely on the microscopic analysis of blood smear images are tedious, slow, and greatly reliant on skilled pathologists. These constraints generate an urgent demand for smart, automated diagnostic tools that can help healthcare providers detect leukemia cells swiftly and accurately.

With the swift progress in deep learning and computer vision, particularly in real-time object detection, there is significant potential to transform the medical imaging sector. The YOLO (You Only Look Once) series of models, recognized for their rapid detection speed and precision, presents an encouraging method to automate leukemia identification in peripheral blood smear images.

1.3 Problem Definition and Objectives

Leukemia cell detection in blood smear images is a challenging and time-consuming task that heavily relies on the expertise of medical professionals. The risk of misinterpretation increases due to variations in image quality, overlapping cells, and subtle differences between leukemia-affected and healthy cells. These challenges underscore the urgent need for an automated, accurate, and real-time diagnostic system to assist healthcare providers in identifying leukemia cells effectively.

Recent advancements in deep learning, particularly in object detection models such as those from the YOLO (You Only Look Once) family, offer promising solutions for automating this task. However, limited comparative research has been conducted to assess the performance of different YOLO variants specifically for leukemia cell detection.

The primary objectives of this project are:

1. To evaluate the performance of YOLOv5, YOLOv8, and YOLOv11 models in detecting leukemia cells from microscopic blood smear images using metrics such as precision, recall, mean Average Precision (mAP), and inference time.
2. To select the most suitable model for real-time deployment based on performance and ease of implementation.
3. To design and develop a Flask-based web application that integrates the selected YOLO model and allows users to upload blood smear images and view detection results in real-time.
4. To assess the effectiveness of the system in terms of detection accuracy, real-time usability, and potential clinical relevance.

1.4 Methodologies of Problem solving

This section outlines the complete pipeline for developing the leukemia cell detection system, from data acquisition and preprocessing to model training and deployment in a webbased interface.

A. Dataset Collection and Annotation

The ALL-IDB1 dataset was selected for its high-resolution microscopic blood smear images. The dataset contains images labeled with either Acute Lym-

phoblastic Leukemia (ALL) or healthy hematogones (HEM). Each image with leukemic cells includes centroid coordinates in the .xyc format. These centroids were converted into bounding boxes using Roboflow, enabling the dataset to be used for object detection tasks.

B. Data Preprocessing and Augmentation

To ensure the dataset was model-ready, the following preprocessing and augmentation techniques were applied:

- All images were resized to 640×640 pixels.
- Pixel normalization scaled all values to the range $[0, 1]$.
- Data augmentation techniques included HSV color shifts, random rotations ($\pm 15^\circ$), brightness and contrast variation, as well as horizontal and vertical flips.
- The dataset was split into 70% training, 20% validation, and 10% testing subsets to ensure fair model evaluation.

C. Model Configuration and Training

Three YOLO variants—YOLOv5, YOLOv8, and YOLOv11—were trained for comparative analysis of detection performance. Each model was trained using PyTorch in a GPU-accelerated environment with the following configuration:

- Epochs: 100
- Batch Size: 16
- Image Size: 640×640
- Learning Rate: 0.01 (YOLOv5), 0.001 (YOLOv8/YOLOv11)
- Optimizer: SGD (YOLOv5), AdamW (YOLOv8/YOLOv11)

Performance was measured using the following metrics:

- Precision and Recall
- Mean Average Precision at IoU thresholds 0.5 (mAP@50) and 0.5:0.95 (mAP@50:95)
- Inference Time per image

D. Backend Development

The YOLOv5 model was deployed using a Flask-based Python server. The backend was designed to:

- Accept image uploads through a RESTful API.
- Perform inference using the trained YOLOv5 model.
- Render results with bounding boxes and class labels on the image.
- Return detection results including the output image path and structured JSON data containing class-wise counts, confidence scores, and bounding box coordinates.

E. Frontend Interface

A simple, lightweight, and user-friendly web interface was developed using HTML, CSS, and JavaScript. Key features include:

- Image upload functionality for users.
- Side-by-side display of original and annotated output images.
- Display of statistics such as detected cell count, confidence scores, and class distribution.
- Interactive UI elements for improved usability and user experience.

F. System Workflow Summary

The end-to-end workflow of the system is summarized as follows:

1. User uploads a blood smear image via the web interface.
2. The Flask backend receives the image and processes it using the YOLOv5 model.
3. Detection results are rendered with bounding boxes and saved.
4. The processed image and detection statistics are returned to the frontend for display.

CHAPTER 2

LITERATURE SURVEY

The architectural evolution from YOLOv8 to YOLOv11 has been explored through extensive analysis of academic journals, official documentation, and source code. YOLOv8 is renowned for its anchor-free detection technology, which excels at small object recognition and multitasking. YOLOv9 introduced a flexible gradient data structure that eliminates information bottlenecks, while YOLOv10 brought NMS-free training, enhancing deployment efficiency, and incorporated a spatial-channel decoupled reduction technique to reduce computational overhead. YOLOv11, which introduced the C3k2 block, has managed to lower the number of parameters while retaining robust feature extraction capabilities [1].

A modified YOLOv5s-based detection algorithm was proposed in SDE-YOLO: A Novel Method for Blood Cell Detection to enhance blood cell detection accuracy. The approach integrates the Swin Transformer for improved feature extraction, optimizes the Path Aggregation Network (PANet) by reducing parameters, and replaces Euclidean Intersection over Union (EIOU) loss with Complete Intersection over Union (CIOU) to enhance convergence. This combination led to significant performance improvements, achieving a mean Average Precision (mAP) of 99.595.3models like SSD, YOLOv4, and YOLOv5s [2].

The comparative performance of YOLOv5nu, YOLOv8n, and YOLOv11n in identifying and classifying red blood cells (RBCs), white blood cells (WBCs), and platelets was evaluated in A Comparative Analysis of YOLOv5nu, YOLOv8n, and YOLOv11n Models for Blood Cell Detection and Classification. YOLOv5nu achieved 96.8while YOLOv8n performed better with 97.395.198.5precise blood cell analysis in medical applications [3].

In the study of GRP-YOLOv5, improvements were made to the regular YOLOv5 model by adding the Gamma transformation and the ResC2Net module. These modifications enhanced the model's ability to capture small features, boosting precision without significantly lowering recall. Additionally, an adaptive anchor box selection method was introduced to minimize misclassification errors. Experimental results demonstrated that GRP-YOLOv5 outperformed the standard YOLOv5 model in terms of F1-score and mAP, making it more suitable for practical flaw detection [4].

In Real-time Detection of Acute Lymphoblastic Leukemia Cells Using Deep Learning, YOLOv5s was implemented for leukemia cell detection. The model achieved an impressive 97.2second (FPS), making it effective for real-time medical diagnostics. The model's high recall ensured minimal false negatives, which is crucial for accurate diagnosis [5].

YOLOv5 is known for its efficiency, accuracy, and speed in real-time object detection. It provides a scalable and adaptable architecture with various model variations (YOLOv5s, m, l, x), making it suitable for a wide

range of applications. It has shown superior performance on benchmark datasets like COCO and PASCAL VOC, outperforming older models like YOLOv4, SSD, and Faster R-CNN. Its applications in real-time systems like autonomous driving, healthcare, and surveillance showcase its versatility [6].

The improvements in YOLOv5, YOLOv8, and YOLOv10 have significantly impacted the architecture and performance of real-time object detection models. YOLOv5 introduced CSPDarknet and mosaic augmentation to balance speed and accuracy, while YOLOv8 added an anchor-free design for better small object detection. YOLOv10 improved on this with large-kernel convolutions, spatial-channel decoupling, and NMS-free inference, enhancing both efficiency and accuracy [7].

The performance of three YOLO-based models—YOLOv5, YOLOv8, and YOLOv11—for the task of identifying leukaemia cells in microscopic blood smear images was assessed and contrasted in this study. YOLOv5 was chosen for real-time deployment because of its lightweight design, ease of integration, and superior recall—all of which are critical in reducing false negatives in medical diagnostics—even though YOLOv11 demonstrated the highest overall accuracy and recall. The viability of using deep learning models for realtime leukaemia detection is demonstrated by the successful integration of YOLOv5 into a Flask-based web application. With room for improvement and clinical adaptation, the results validate the efficacy of YOLO-based detectors in facilitating automated, early, and efficient leukaemia diagnosis. [8].

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

Assumptions

- For model inference via the Flask web server, the system assumes that a dependable internet connection will be available.
- Users will upload photographs of microscopic blood smears in compatible file types, such as .jpg and .png.
- The client system will have a modern web browser capable of supporting HTML5 and JavaScript.
- The Flask backend server will have sufficient GPU or CPU power to carry out inference jobs efficiently.

Dependencies

- The ALL-IDB1 dataset's accessibility is essential to the project's success.
- For backend operations, the system requires the Flask framework in addition to necessary Python modules like PyTorch, NumPy, and OpenCV.
- To perform cell detection, YOLOv5 weights and configuration files are necessary.
- The dataset is labeled and arranged using Roboflow.

3.2 Functional Requirements

3.2.1 Uploading and Image Detection System Capability 1

- It must be possible for users to upload blood smear images through a web interface.
- In order to identify leukemia cells, the uploaded image will be sent to the backend.
- Annotated images with bounding boxes and class labels indicating detected cells will be generated by the system.

3.2.2 System Aspect 2: Results Visualization

- For easy comparison, the system will display the original and processed photos side by side.
- The system will provide statistical information, including the number of cells found, the distribution of classes, and the confidence levels for each cell found.

3.2.3 System Features: Accessibility and User Involvement

- The system will include an interactive, user-friendly interface that reacts instantly to user input.
- The experience is made more efficient by allowing users to reset and re-upload photographs without having to completely reload the website.

3.3 External Interface Requirements

3.3.1 User Interfaces

Using common web technologies like HTML5, CSS, and JavaScript, the system has a web-based interface. Through this tool, users can directly upload photos of microscopic blood smears for analysis. The interface displays the original and processed photos with bounding boxes and class labels highlighted once the analysis is complete. Important statistics including the number of cells found, class breakdown, and confidence levels are also included. Both technical and non-technical users will find the interface to be responsive and easy to use.

3.3.2 Hardware Interfaces

- The system is compatible with general-purpose devices such as laptops, desktop computers, and tablets with internet connectivity.
- The backend server must be equipped with a GPU to perform efficient image processing and inference.
- On the client side, only a modern web browser with support for JavaScript and HTML5 is required.

3.3.3 Software Interfaces

- **Operating System:** The system supports cross-platform environments including macOS, Linux, and Windows.
- **Jupyter Notebook:** Used during development and model experimentation phases.
- **Dataset:** The ALL-IDB1 dataset is used for training and evaluation.
- **Programming Language:** Python is the primary language for model development and backend integration.
- **Frameworks:** Flask for web server operations, PyTorch for model inference, and OpenCV for image processing.

3.3.4 Communication Interfaces

- RESTful API uses the HTTP protocol for communication between the client and server.
- JSON is the primary data exchange format used between the frontend and backend.
- The system is deployed on a cloud platform and communicates securely using HTTPS.
- Real-time feedback on image processing and inference results is provided through asynchronous responses.

3.4 System Requirements

3.4.1 Database Requirements

The system utilizes the publicly available ALL-IDB1 dataset for training and evaluation. This dataset includes high-resolution microscopic blood smear images labeled with centroid coordinates for leukemia (ALL) and healthy cells (HEM). No external database system is required as all data handling is performed locally using file-based storage.

3.4.2 Software Requirements (Platform Choice)

- **Operating System:** Windows 10 / Linux (Ubuntu 20.04)
- **Programming Language:** Python 3.9
- **Frameworks and Libraries:**
 - PyTorch for model training
 - Flask for web application backend
 - HTML, CSS, and JavaScript for frontend
 - Roboflow for annotation and bounding box conversion

3.4.3 Hardware Requirements

- **CPU:** Intel Core i7 or equivalent
- **GPU:** NVIDIA RTX 3060 or higher

- **RAM:** Minimum 16 GB
- **Storage:** 50 GB free space for dataset, models, and output results

3.5 Project Development Process

This project uses the Agile Model of Software Development Life Cycle (SDLC) because it offers feedback flexibility and iterative development approaches, which proved beneficial for the project. Through the Agile methodology, the project maintained quick prototyping with constant improvement steps from start to finish. The following phases were executed:

- **Project Planning:** Project planning involved the use of the ALL-IDB1 dataset alongside YOLOv5, YOLOv8, and YOLOv11 models for leukemia cell detection.
- **Numerical Evaluation of Models:** Numerical evaluation of multiple chosen models involved determining precision values together with recall statistics, mAP values, and inference time measurements.
- **Web Application Development:** The development of the Flask-based web application received architectural attention through design work for both frontend and backend sections.
- **Model Training:** The project team executed model training with PyTorch before creating APIs in Flask and frontend interfaces using HTML, CSS, and JavaScript.
- **Evaluation of Detection Accuracy:** The detection accuracy, together with the system functionality, was evaluated by conducting real-time image tests.
- **Web-based Deployment:** The YOLOv5 model undergoes web-based deployment as part of the interactive application for user engagement.
- **Future Versions:** Future versions of the system will include maintenance features, which support the update of model versions alongside the expansion of the accessible datasets.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

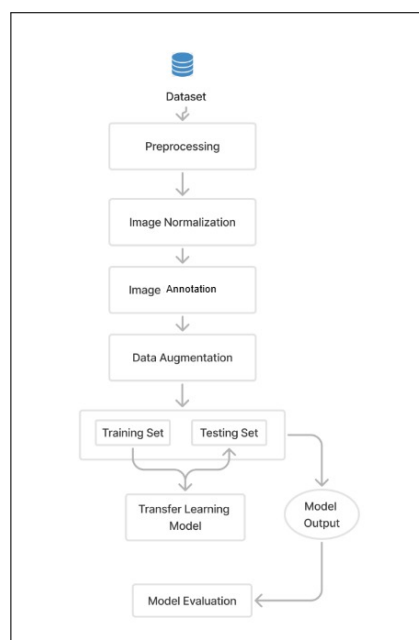


Figure 4.1: Model Architecture

4.2 Data Flow Diagrams

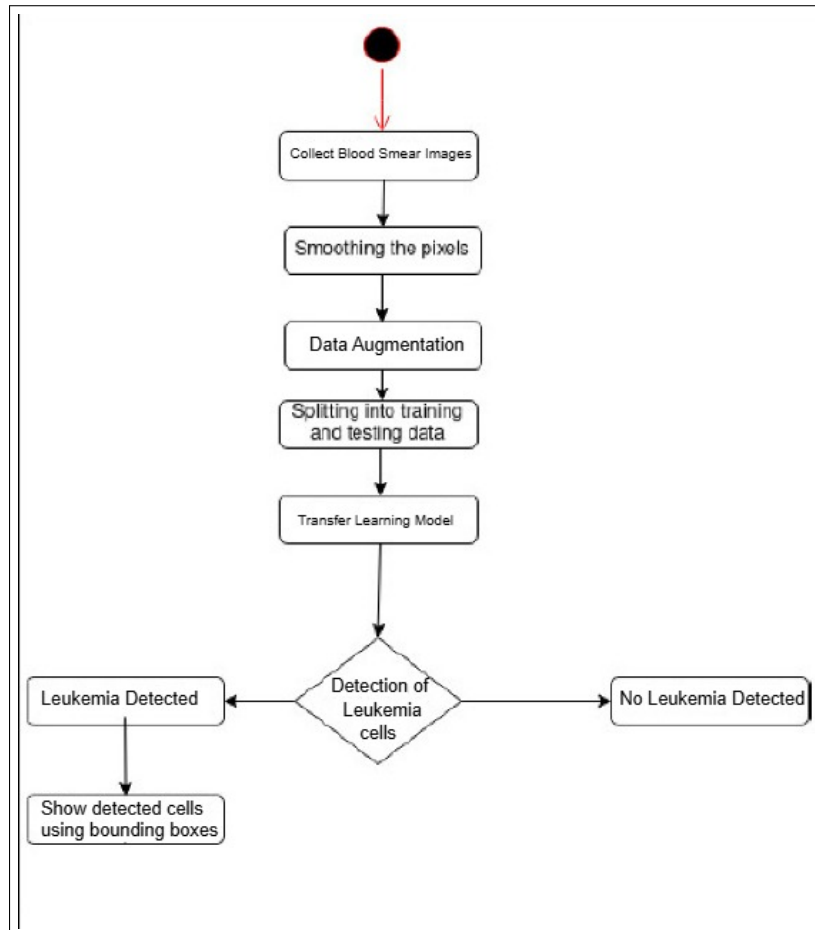


Figure 4.2: Data Flow Diagram

4.3 UML Diagrams

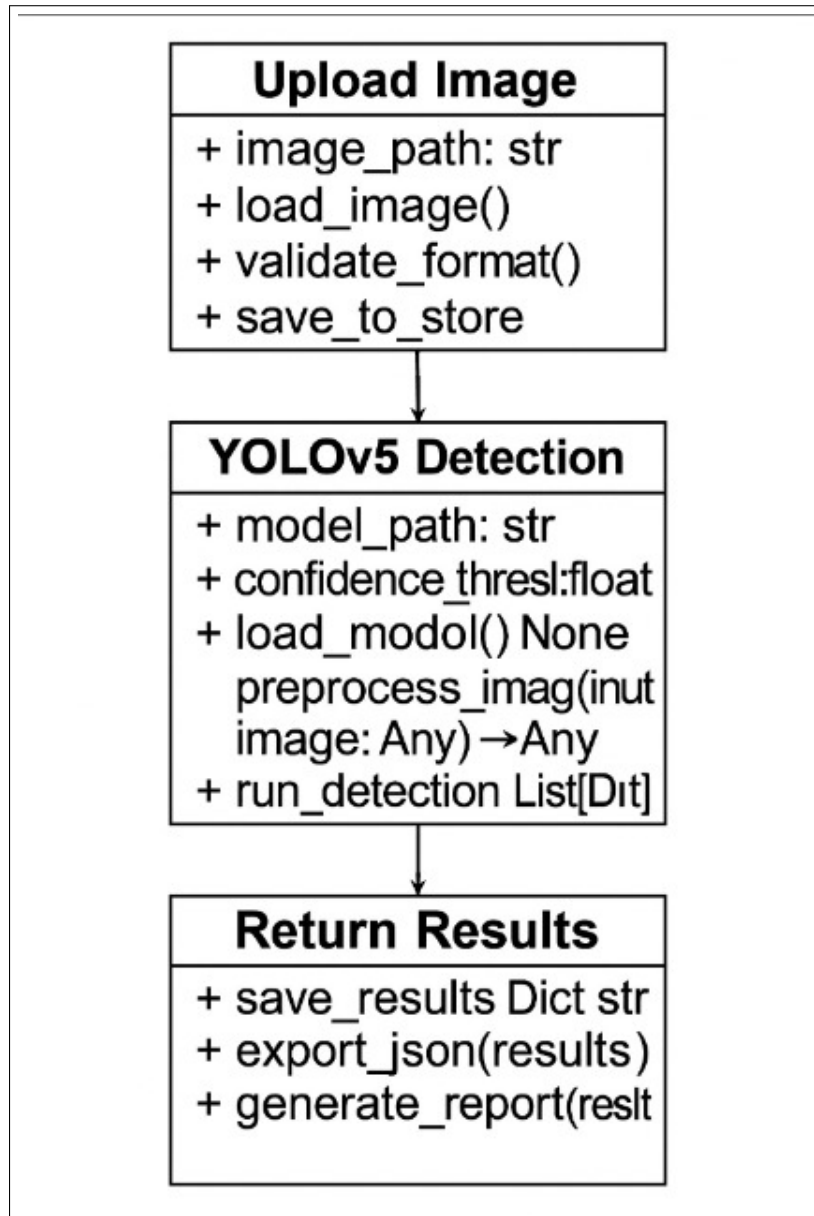


Figure 4.3: UML Diagram

4.4 Sequence Diagram

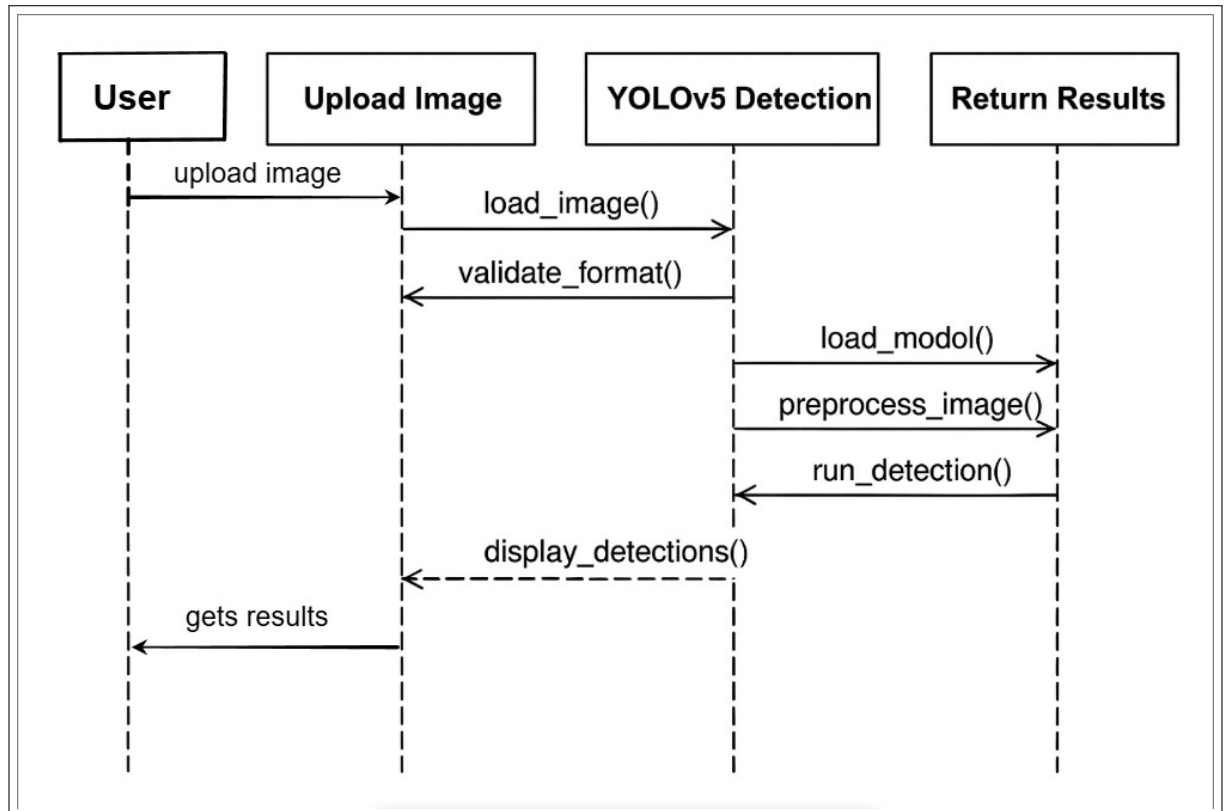


Figure 4.4: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Data Collection Cost:** The cost associated with collecting and managing the data required, including data acquisition, storage, preprocessing, and ongoing maintenance.
- **False Positive Cost:** The cost associated with predicting a positive outcome when the actual outcome is negative.
- **False Negative Cost:** The cost associated with predicting a negative outcome when the actual outcome is positive.
- **Accuracy Cost:** The overall accuracy of the prediction model and its impact on the effectiveness of the system. A higher accuracy indicates a better ability to correctly predict cases, reducing both false positives and false negatives.
- **User Acceptance Cost:** The level of acceptance and usability of the system by healthcare professionals, patients, and stakeholders.

5.1.2 Project Resources

1. **Software Developers:** Responsible for designing, developing, and optimizing the software and features of the application.
2. **AI Developers:** Responsible for development of the RAG pipeline for test case generation from documents.
3. **Project Management Tool:** *Jira* will be utilized for tracking project tasks, managing workflows, and facilitating communication among team members. It will help in organizing sprints, monitoring progress, and ensuring that the project stays on schedule.
4. **Collaboration Tools:** *GitHub* for version control and collaborative coding, *Google Colab* for sharing development notes and prototypes, *Google Meet* for team meetings and discussions, and *Discord* for real-time communication and collaboration among team members.

5.2 Risk Management

5.2.1 Risk Identification

Risk management in leukemia classification using deep learning involves identifying and addressing potential challenges to project success. Key risks include:

- **Data Quality Issues:** Biases or incomplete datasets can impact model accuracy.
- **Overfitting:** Requires regularization techniques and cross-validation to mitigate.
- **Limited Computational Resources:** May hinder model training and testing, necessitating scalable infrastructure solutions.
- **Regulatory Compliance and Ethical Considerations:** Ensuring patient data privacy and legal adherence is critical.

Proactive risk management strategies, including regular assessments, contingency plans, and stakeholder communication, are crucial for mitigating risks and achieving project objectives.

5.2.2 Risk Analysis

Once risks are identified, a risk analysis will be conducted to assess their likelihood and potential impact. This includes:

- Evaluating risks based on probability, severity, and potential consequences.
- Ranking risks by their potential impact on budget, operations, and schedule.
- Developing contingency plans for high-priority risks.
- Conducting regular risk assessments throughout the project lifecycle.

5.2.3 Overview of Risk Mitigation, Monitoring, and Management

- **Technical Risk Mitigation:** Regular performance testing and code reviews will be conducted to identify and resolve technical issues early in the development process for leukemia classification using deep learning.
- **Data Quality Risk Mitigation:** A comprehensive data validation process will be implemented to ensure that datasets used for training and testing are accurate and representative, reducing biases and enhancing model reliability.
- **Overfitting Risk Mitigation:** Regularization techniques and cross-validation will be employed to minimize overfitting and ensure that the model generalizes well to unseen data.
- **Resource Availability Risk Mitigation:** Scalable infrastructure solutions will be established to accommodate the computational requirements of deep learning models, ensuring that sufficient resources are available throughout the project lifecycle.
- **Regulatory Compliance and Ethical Considerations:** Ongoing assessments will be conducted to ensure compliance with legal standards for patient data privacy, and ethical guidelines will be followed to address any concerns related to data usage.

5.3 Project Schedule

5.3.1 Project Task Set

- **Data Acquisition:** Gather a diverse dataset of high-resolution blood smear images from microscopic examinations, covering various types and subtypes of blood cancer.
- **Data Preprocessing:** Clean and preprocess the dataset to enhance image quality, standardize features, and ensure consistency across samples. Tasks include noise reduction, contrast enhancement, and normalization.
- **Model Selection and Architecture Design:** Choose appropriate deep learning architectures such as Convolutional Neural Networks (CNNs), and design the network considering factors like depth, kernel sizes, and activation functions.
- **Feature Extraction:** Implement deep learning-based automatic feature extraction from preprocessed images to capture relevant patterns and structures indicative of blood cancer.
- **Transfer Learning Implementation:** Use pre-trained CNN models and adapt them to the blood cancer classification task to enhance performance.
- **Model Training:** Train the model with the preprocessed and annotated dataset, adjusting hyperparameters and optimizing algorithms for accuracy and robustness.
- **Model Evaluation:** Evaluate performance using accuracy, sensitivity, specificity, and F1-score. Validate generalizability through cross-validation and independent dataset testing.
- **Model Deployment:** Deploy the trained model for real-time inference, integrating it into clinical workflows for diagnostic support.
- **Error Analysis:** Analyze model errors and misclassifications to uncover discrepancies and guide further model improvements.

5.3.2 Task Network

Figure 5.1: Project Timeline Chart

The following table outlines the timeline of key tasks from project inception to completion.

SR NO.	TASK	LABOUR (HOURS/DAYS)
1	Project Planning and Preparation	1 month
2	Studying the Scope and Feasibility	2 weeks
3	Data Acquisition and Preprocessing	1 month
4	Model Development and Training	2 months
5	Model Evaluation and Optimization	1 month
6	Software Development and Integration	5 weeks
7	Testing and Quality Assurance	2 weeks
8	Documentation and Presentation	2 weeks
9	Final Review	1 week

5.4 Team Organization

5.4.1 Team Structure

- **Data Collection and Annotation Team:** Responsible for gathering blood smear images and annotating them with ground truth labels. This team ensures the dataset's quality and diversity to support effective model training.
- **Model Development Team:** Designs and implements machine learning and deep learning models for blood cancer classification. They focus on optimizing model architectures, hyperparameters, and training procedures.
- **Software Development Team:** Builds the software infrastructure required for data preprocessing, model training, and deployment. This team also develops user interfaces for visualizing data, evaluating models, and interpreting results.

5.4.2 Management Reporting and Communication

- Project progress will be monitored through regular team meetings and status updates.
- Communication between teams will be facilitated using project management and collaboration tools such as Jira, GitHub, Google Meet, and Discord.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- **Dataset Annotation and Collection:** The ALL-IDB1 dataset was utilized, and centroid annotations were converted into bounding boxes to make the data suitable for object detection tasks.
- **Data Preprocessing and Augmentation:** Images were resized and normalized, with enhancements applied using HSV color shifts, rotations, and flips. The dataset was split into training, validation, and testing subsets to ensure balanced model evaluation.
- **Model Configuration and Training:** YOLOv5, YOLOv8, and YOLOv11 models were trained using different learning rates and optimizers to compare their performance based on precision, recall, and mean Average Precision (mAP).
- **Backend Development:** A Flask-based backend was developed to support the YOLOv5 model. It facilitates image uploads and returns detection results in JSON format for further processing.
- **Frontend Interface:** Developed using HTML, CSS, and JavaScript, the interface enables intuitive image uploads, displays the original and annotated images side-by-side, and provides statistics like cell count and confidence scores.
- **System Workflow:** The complete workflow includes image upload via the frontend, backend inference using the trained model, rendering of detection results, and displaying processed outputs on the frontend, ensuring a smooth end-to-end user experience.

6.2 Tools and Technologies Used

- **Programming Languages:** Python, HTML, CSS, JavaScript
- **Frameworks and Libraries:** PyTorch, Flask, Roboflow, OpenCV
- **Models:** YOLOv5, YOLOv8, YOLOv11
- **Web Technologies:** RESTful API, JSON for communication between frontend and backend
- **Deployment Environment:** GPU-accelerated system for training, web server for deployment

6.3 Algorithm Details

6.3.0.1 YOLOv5

YOLOv5 is among the most widely used iterations of the YOLO (You Only Look Once) object detection framework. It is recognized for its speed, minimal weight, and ease of deployment. These attributes render it perfect for real-time applications, even on devices with constrained resources.

Mosaic Augmentation: This method merges four distinct images into a single one during the training process. It enhances diversity in training data, aiding the model in generalizing more effectively.

CIoU Loss: This is an intelligent approach to assess the alignment of predicted boxes with the actual locations of objects. It takes into account shape, size, and distance, rather than merely overlap.

Reason for use in this research: YOLOv5 demonstrated high recall (i.e., it identified most of the real objects), which is crucial in medical contexts where overlooking even one abnormal cell could have severe consequences.

6.3.0.2 YOLOv8

YOLOv8 is a newer and enhanced iteration that introduces numerous enhancements compared to YOLOv5.

Anchor-Free Detection: Unlike previous models that required pre-determined "anchor boxes" for estimating object dimensions, YOLOv8 predicts object positions directly. This improves its ability to locate small or overlapping cells.

Enhanced Accuracy: YOLOv8 excels at reducing false positives—it seldom classifies something as unusual when it is not.

More Adaptable Structure: It employs modern elements that enhance both precision and efficiency in various activities.

6.3.0.3 YOLOv11

YOLOv11 is the latest and most sophisticated edition of the three. It seeks to enhance both precision and detection capability, particularly in intricate surroundings.

Attention Mechanisms: These help the model focus on the most important parts of the image—like a human paying closer attention to suspicious regions in a microscope image.

Highest Performance: During testing, YOLOv11 had the best accuracy and recall, meaning it found almost all the abnormal cells.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

- **Unit Testing:** To ensure that components function as intended in isolation, individual parts such as model inference, preprocessing functions, and Flask API routes were thoroughly tested.
- **Integration Testing:** Verified the interaction between the Flask backend and the frontend interface. This included image uploads, model invocation, and rendering of detection output.
- **System Testing:** The complete web application was tested to ensure it met both functional and non-functional requirements, including inference speed, UI responsiveness, and detection accuracy.
- **Performance Testing:** Focused on evaluating the model's inference time, particularly for real-time detection. YOLOv5 was tested to ensure consistent operation within 15 milliseconds per image.
- **Usability Testing:** Ensured that the web interface was intuitive and user-friendly, enabling even non-technical users to interact effectively with the system.

7.2 Test Cases & Test Results

- **Test Case 1: Image Upload Capabilities**
Goal: Check whether users can upload blood smear images via the web interface.
Result: Passed – Images were successfully uploaded and displayed on the frontend.
- **Test Case 2: Model Inference and Prediction Display**
Goal: Verify that uploaded images are processed appropriately and detection results are returned and displayed.
Result: Passed – Predictions including bounding boxes and class labels were accurately rendered on output images.
- **Test Case 3: Inference Time**
Goal: Ensure that detection occurs in real-time (under 15 ms).
Result: Passed – Average inference times were 13.1 ms (YOLOv11) and 14.2 ms (YOLOv5/YOLOv8).

CHAPTER 8

RESULTS

8.1 Outcomes

1. **YOLOv5 Outperformed Other Models for ALL Detection:** Among YOLOv5, YOLOv8, and YOLOv11, YOLOv5 provided the most reliable results in detecting Acute Lymphoblastic Leukemia (ALL) cells. Its high recall rate significantly reduced false negatives, which is vital in medical diagnosis.
2. **YOLOv5 Chosen for Real-Time Deployment:** Due to its lightweight architecture, faster inference speed, and superior recall, YOLOv5 was deployed in a Flask-based web application. It ensured accurate and efficient leukemia detection, suitable even on lower-end systems.
3. **YOLOv8 and YOLOv11 Showed High Accuracy, But Lower Practicality:** Although YOLOv11 achieved the highest accuracy and YOLOv8 provided a balance between speed and accuracy, both were less optimal for real-time clinical use due to integration complexity and hardware requirements.

8.2 Screen Shots

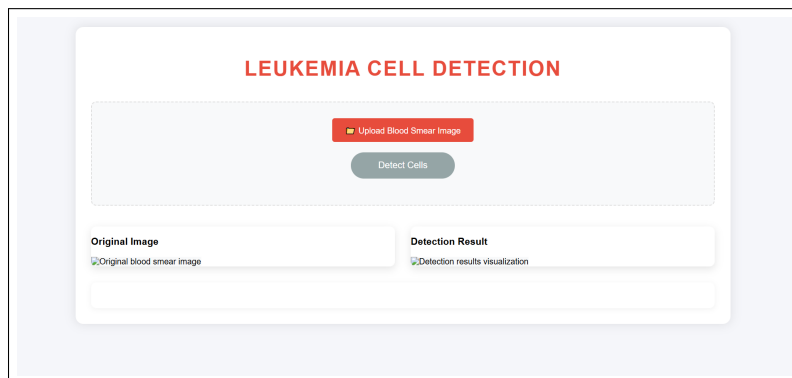


Figure 8.1: Screenshot 1: Image Upload Interface

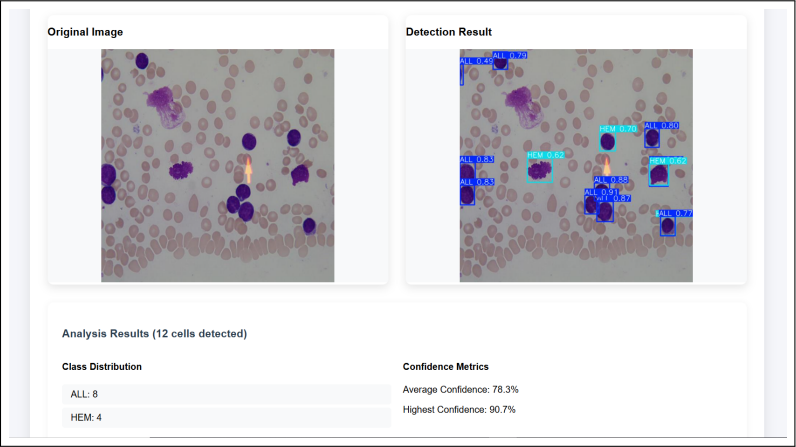


Figure 8.2: Screenshot 2: Detection Results with Bounding Boxes



Figure 8.3: Screenshot 3: Detection Statistics and Insights

CHAPTER 9

CONCLUSIONS

9.1 Conclusion

Three YOLO-based models—YOLOv5, YOLOv8, and YOLOv11—that are used to identify leukaemia cells in microscopic blood smear images are described and their architectures compared in this study. The latest development in the YOLO series, YOLOv11, achieves the highest overall accuracy by introducing enhanced feature extraction and attention mechanisms. With its modular design, YOLOv8 provides a fair compromise between speed and accuracy. But YOLOv5, which is renowned for its lightweight design and simplicity of use, beat the others in terms of recall for identifying ALL (acute lymphoblastic leukaemia) cells. This makes it particularly useful in reducing false negatives, which are a critical component of medical diagnosis. It was chosen for real-time deployment via a Flask-based web application due to its exceptional recall and dependable performance. The integration's success validates that using deep learning-based object detection systems for early, automated, and efficient leukemia detection, with scope for future clinical adaptation and enhancement.

9.2 Future Work

1. To further increase detection accuracy and decrease inference time, the system can be improved in the future by incorporating more sophisticated YOLO variants or transformer-based architectures.
2. Model generalisation and robustness across wider clinical scenarios may be enhanced by enlarging the dataset to include more uncommon and varied leukaemia types.
3. By combining segmentation models with object detection, pathologists may be able to locate cell boundaries precisely, which would help with diagnosis.
4. In healthcare settings with limited resources, the system's usability and accessibility can be improved by deploying it on mobile platforms or edge devices.
5. To allow for ongoing model improvement and adaptation to actual clinical feedback, a feedback loop from subject matter experts can be included.

9.3 Applications

1. **Clinical Diagnostics:** The system can help medical professionals identify leukaemia early from blood smear images in pathology labs and hospitals.
2. **Telemedicine:** Healthcare delivery in underserved or rural areas can be supported by integration with remote diagnostic platforms.
3. "Medical Education:" can be used as a teaching tool to help professionals and students comprehend and identify leukaemia cell patterns in microscopic images.
4. **Research and Development:** aids in the creation and verification of AI-based diagnostic instruments for research on medical imaging and haematology.
5. **Programs for Health Screening:** beneficial for automated, mass screening at health camps or government-sponsored health programs.

CHAPTER 10

APPENDIX

Appendix A

Problem Statement and Feasibility Assessment

The project focuses on the detection of leukemia cells, particularly Acute Lymphoblastic Leukemia (ALL), using deep learning-based object detection models. The primary objective is to deploy a system capable of performing accurate real-time analysis on blood smear images.

Computational Complexity Consideration

The feasibility of this problem is linked to its classification in computational complexity theory:

- The underlying task—image classification and object detection—is in class **P** (Polynomial time), as deep learning inference runs in polynomial time with respect to input image size.
- However, optimizing model architecture or training deep models may approach NP-Hard complexity, especially for hyperparameter tuning or model pruning tasks.
- The satisfiability of detection results (e.g., matching predicted vs. actual cell locations) can also be analyzed using constraint satisfaction or logic-based formulations.

Dataset Details

- **Dataset Used:** ALL-IDB1 (Acute Lymphoblastic Leukemia Image Database).
- **Format:** High-resolution microscopic images labeled with centroid annotations converted to bounding boxes.
- **Split:** 70% Training, 20% Validation, 10% Testing.

Experimental Setup

- **Framework:** PyTorch
- **Hardware:** GPU-accelerated environment
- **Image Size:** 640×640

- **Training Epochs:** 100
- **Batch Size:** 16
- **Optimizers:** SGD (YOLOv5), AdamW (YOLOv8, YOLOv11)

Performance Metrics

- **Precision and Recall:** Evaluates false positives and false negatives.
- **mAP@50 and mAP@50:95:** Mean average precision at varying IoU thresholds.
- **Inference Time:** Speed per image during real-time detection.

YOLO Model Summary

- **YOLOv5:** Highest recall, chosen for deployment due to real-time speed and integration simplicity.
- **YOLOv8:** Balanced precision and speed.
- **YOLOv11:** Highest overall accuracy but complex to deploy.

Tools and Libraries Used

- Python, PyTorch, Flask, OpenCV
- Roboflow for annotation and preprocessing
- HTML, CSS, JavaScript for frontend UI

Appendix B

Title

Comparative Evaluation of YOLOv5, YOLOv8, and YOLOv11 for Real-Time Leukemia Cell Detection in Blood Smear Images

Name of Conference

International Research Journal of Engineering and Technology (IR-JET)

Certificate

Attached as Appendix B.1

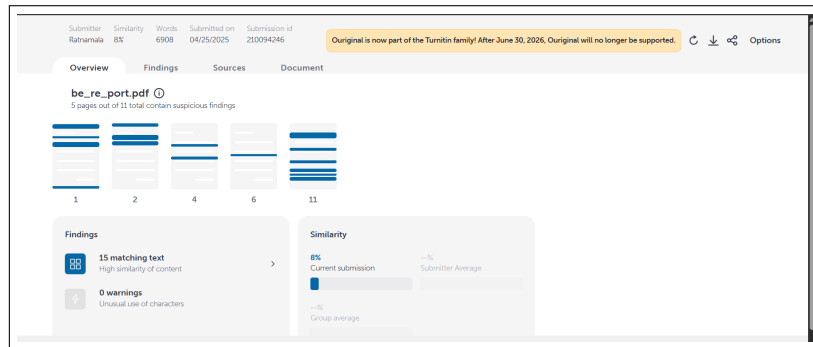
Published Paper

Attached as Appendix B.2

Additional Documentation

All documentation related to the publication, including screenshots of the submission portal, certificates, and acceptance letters, are provided in the annexed pages.

10.1 Appendix C



Thus, the overall plagiarism percentage is 8 percent.

CHAPTER 11

REFERENCES

- [1] P. Hidayatullah, N. Syakrani, M. R. Sholahuddin, T. Gelar, and R. Tubagus, "YOLOv8 to YOLO11: A Comprehensive Architecture In-depth Comparative Review," Preprint, Jan. 23, 2025.
- [2] Y. Wu, D. Gao, Y. Fang, X. Xu, H. Gao, and Z. Ju, "SDE-YOLO: A Novel Method for Blood Cell Detection," *Biomimetics*, vol. 8, no. 5, p. 404, Sep. 2023, doi: 10.3390/biomimetics8050404.
- [3] T. Li, "A Comparative Analysis of YOLOv5nu, YOLOv8n, and YOLOv11n Models for Blood Cell Detection and Classification," in *Proc. 5th Int. Conf. on Signal Processing and Machine Learning (SPML)*, 2025.
- [4] Y. Zhao, B. Chen, B. Liu, C. Yu, L. Wang, and S. Wang, "GRPYOLOv5: An Improved Bearing Defect Detection Algorithm Based on YOLOv5," *Sensors*, vol. 23, no. 17, p. 7437, Aug. 2023.
- [5] E. Chen, R. Liao, M. Y. Shalaginov, and T. H. Zeng, "Real-time Detection of Acute Lymphoblastic Leukemia Cells Using Deep Learning," in *Proc. IEEE Conf. on Biomedical Imaging (CBMI)*, 2025. (Exact conference name to be confirmed).
- [6] S. K. Jaiswal and R. Agrawal, "A Comprehensive Review of YOLOv5: Advances in Real-Time Object Detection," in *Proc. BN College of Engineering and Technology, Dept. of Computer Science and Engineering*, Lucknow, India, accepted May 7, 2024.
- [7] M. Hussain, "YOLOv5, YOLOv8 and YOLOv10: The Go-To Detectors for Real-Time Vision," *arXiv preprint*, arXiv:2407.02988, Jul. 3, 2024. [Online].
- [8] X. Song and H. Tang, "Blood Cell Target Detection Based on Improved YOLOv5 Algorithm," *Electronics*, vol. 13, no. 4992, Dec. 2024. [Online].
- [9] L. Du, R. Zhang, and X. Wang, "Overview of Two-Stage Object Detection Algorithms," *J. Phys.: Conf. Ser.*, vol. 1544, p. 012033, 2020, 5th Int. Conf. on Intelligent Computing and Signal Processing (ICSP), Suzhou, China. doi: 10.1088/1742-6596/1544/1/012033.